

CSE 401 – Numerical Methods

CT3: Theoretical Questions with Answers

Strictly from NZR (Suchi) hand-notes – nothing omitted

September 8, 2026

How to use this. Every conceptual point in sir's notes is turned into a question with a model answer. Questions the notes explicitly flag as likely exam questions are marked **[FLAGGED IN NOTES]**. Work through them all — sir said to follow his notes exactly.

Contents

1	Golden Section Search	1
2	Gradient Descent & Linear Regression	2
3	Newton's Method for Optimization	3
4	Foundations of Optimization	4
5	Constrained Optimization (Lagrange Multipliers)	5
6	Least-Squares Regression & Correlation	6
7	Multiple Linear Regression	7
8	Lagrange Interpolation	8
9	Linear Interpolation	9
10	Newton's Divided-Difference Interpolation (v2 notes)	9
11	Spline Interpolation (v2 notes)	10

1 Golden Section Search

Q1.1. What is Golden Section Search and what is it used for?

Ans. It is an **optimization technique** — an efficient way to locate the extremum (maximum or minimum) of a single-variable, unimodal function. It is directly related to the Golden Ratio, which is used to place the search points so that function evaluations are reused across iterations.

Q1.2. What assumption must hold for the method to work, and when does it fail?

Ans. It assumes there is **exactly one extremum** in the bracketing interval $[x_L, x_u]$ (the function is unimodal there). It **fails if there are multiple extrema** in the interval, because the “keep the better side” logic can then discard the region containing the true optimum.

Q1.3. Why is it called “golden”, i.e. what is the efficiency advantage?

Ans. The interior points are placed using the Golden Ratio $R = \frac{\sqrt{5}-1}{2} \approx 0.618$. This placement makes one of the two new points of the next iteration coincide with a point already evaluated. So the **first iteration needs 2 interior function evaluations**, but **every subsequent iteration needs only 1 new function call**. This minimizes the number of (possibly expensive) function evaluations — the whole purpose of the method is to cut down the number of iterations and function calls.

Q1.4. How do you decide which side contains the maximum?

Ans. Compare the two interior function values: if $f_1 > f_2$ the maximum lies to the **left** (so drop the right part, $x_u = x_2$); if $f_1 < f_2$ it lies to the **right** (drop the left part, $x_L = x_1$). The remaining old interior point is reused as one of the new interior points, carrying its stored f value.

Q1.5. How is the final optimum reported and when do you stop?

Ans. Report the optimum as the midpoint $x_{\text{opt}} = \frac{x_u + x_L}{2}$, $f_{\text{opt}} = f(x_{\text{opt}})$. Stop when $\left| 2 \frac{x_u - x_L}{x_u + x_L} \right| < \varepsilon_{\text{tol}}$ (the relative interval width falls below tolerance).

Q1.6. [FLAGGED IN NOTES] Derive / analyse the Golden Ratio.

Ans. Let $\ell_0 = x_u - x_L$, $\ell_1 = x_2 - x_L$, $\ell_2 = x_u - x_2$. To make the next iteration's points land on old points we require (i) $\ell_0 = \ell_1 + \ell_2$ and (ii) $\ell_1/\ell_0 = \ell_2/\ell_1$. Substituting (i) into (ii) gives $1 + R = 1/R$ with $R = \ell_2/\ell_1$, i.e. $R^2 + R - 1 = 0$, so $R = \frac{-1 \pm \sqrt{5}}{2}$; the positive root is $R = \frac{\sqrt{5}-1}{2} \approx 0.618$. (Full algebra is in the Maths & Proofs booklet.)

2 Gradient Descent & Linear Regression

Q2.1. Name things that different ML methods optimize, and what single tool handles them all.

Ans. Linear regression optimizes **slope and intercept**; logistic regression optimizes the “squiggle” (its slope/intercept); t-SNE / clustering optimize the **cluster** arrangement. **Gradient Descent** is one general tool that can optimize all of these (slope, intercept, cluster, squiggle).

Q2.2. What is the loss function used in linear regression, and why square the residuals?

Ans. The **Sum of Squared Residuals**, $SSR = \sum r_i^2$ with $r_i = y_i - (\text{intercept} + \text{slope} \cdot x_i)$. Residuals are squared so that positive and negative errors do **not cancel out**. Plotting SSR against the parameter gives a convex bowl whose bottom is the best value.

Q2.3. Why use Gradient Descent instead of simply setting the derivative to zero?

Ans. For an optimum we need where the slope = 0. Setting the derivative to zero analytically works for small problems, but often we **cannot solve slope= 0 in closed form** (too many parameters / huge data). GD instead takes steps from an initial guess: it does only a few cheap calculations far from the optimum and progressively finer ones as it gets closer, until it reaches the best value.

Q2.4. Write the GD update and define the step size. What is a “schedule”?

Ans. Step size = $\frac{d}{d(\text{param})}(\text{loss}) \times \text{learning rate}$; and New param = Old param – step size. The **schedule** is the way the learning rate is changed from relatively large (early) to relatively small (later).

Q2.5. What are the stopping/convergence conditions for GD?

Ans. Stop when (i) the **step size is close to zero**, or (ii) a **maximum number of steps** is reached.

Q2.6. Give the steps of GD for multiple parameters (slope & intercept together).

Ans. (1) Take the derivative of the loss w.r.t. each parameter; (2) pick a random value for each parameter (e.g. $i = 0, s = 1$); (3) plug current values into the derivatives; (4) step size = derivative $\times$ learning rate for each parameter; (5) new param = old param – step size; (6) repeat until convergence (step size = 0 or max steps).

Q2.7. What is the problem with plain GD on very large problems, and the solution?

Ans. With millions of parameters/samples, using the whole dataset every step is too expensive (data has redundancy). Solution: **Stochastic Gradient Descent (SGD)** — randomly select a subset of data at every step.

Q2.8. Distinguish strict SGD, mini-batch, and give SGD’s advantages.

Ans. Strict definition: use only **one** sample per step. **Common practice:** use a small subset (**mini-batch**) each step. **Advantages:** (i) best of both worlds; (ii) more stable estimates in fewer steps. **Cool thing:** when new data arrives we don’t need to start from scratch.

3 Newton's Method for Optimization

Q3.1. How does the order of information differ between root finding and optimization?

Ans. **Root finding** uses first-order information (uses f'); **optimization** uses second-order information — it uses the **gradient and the Hessian** (2nd order).

Q3.2. Write the Newton iterations for zero-finding and for optimization.

Ans. Zero-finding: $x_{t+1} = x_t - \frac{f(x_t)}{f'(x_t)}$. Optimization (find where $f' = 0$): $x_{t+1} = x_t - \frac{f'(x_t)}{f''(x_t)}$; vector form $x_{t+1} = x_t - H^{-1}g$. Geometrically it makes a **second-order (quadratic) approximation** and jumps to its vertex.

Q3.3. State the two limitations of Newton's method.

Ans. (1) It finds **any stationary point** (derivative = 0) — local minimum, local maximum, or **saddle point** — and cannot guarantee the global optimum; a saddle/max has zero slope but is not a minimum. (2) It can move **toward a maximum instead of a minimum** if started in the wrong place, because the raw update doesn't check the sign of f'' ; remedy is to use GD steps back and forth / check the curvature direction.

Q3.4. State the three advantages of Newton's method ("why not just use GD?").

Ans. (1) **Faster than GD** (fewer iterations); (2) **not sensitive to the learning rate** (no learning-rate tuning); (3) the number of **correct digits doubles** each step (quadratic convergence $\Rightarrow$ very high accuracy quickly).

Q3.5. [FLAGGED IN NOTES] When do we use GD and when Newton's method?

Ans. Use **GD** when the problem size is large — typical ML problems, where the Hessian is huge/expensive to form and invert. Use **Newton's method** when the problem is small, so the Hessian is cheap, and you gain fast quadratic convergence.

Q3.6. What smoothness condition must f satisfy for Newton's method, and what is the underlying idea in one line?

Ans. $f : \mathbb{R}^n \rightarrow \mathbb{R}$ must be **sufficiently smooth** (continuous derivatives exist); otherwise the method fails. The idea: "**make a second-order approximation and minimize that**" (via Taylor's theorem).

Q3.7. What is the condition for the stationary point to be a minimum?

Ans. The Hessian must be at least **positive semi-definite (PSD)** — every eigenvalue real and ≥ 0 . If every eigenvalue is > 0 it is **positive definite (PD)**. Also: if f is convex, H is PSD.

Q3.8. State the algorithm and the practical issues / fixes.

Ans. Algorithm: initialize $x_0 \in \mathbb{R}^n$; iterate $x_{t+1} = x_t - H^{-1}g$ with $H = \nabla^2 f(x_t)$, $g = \nabla f(x_t)$; stop when the gradient is 0 (i.e. $x_{t+1} = x_t$). **Issues:** (i) H may fail to be PD, so H^{-1} can't be computed — switch toward GD (**Levenberg–Marquardt** algorithm); (ii) rather than invert H , **solve $Hy = g$ for y** then use $x_{t+1} = x_t - y$; (iii) sometimes a **convergence-rate controller** $\alpha_t > 0$ is added: $x_{t+1} = x_t - \alpha_t H^{-1}g$ to converge faster/slower.

4 Foundations of Optimization

Q4.1. What does the gradient represent?

Ans. The gradient returns a **vector field** giving the **direction and magnitude of the steepest slope** for every possible input combination.

Q4.2. What are the two steps that form the basis of all optimization?

Ans. (1) **Set up an objective function**; (2) **solve/search for locations of zero gradient**.

Q4.3. For $J = x^3$, the derivative is zero at $x = 0$ — is it a minimum? Explain.

Ans. No. $J' = 3x^2 = 0$ at $x = 0$, but $J'' = 6x$ changes sign with x , so $x = 0$ is a **saddle/inflection point**, neither a minimum nor a maximum. This shows a zero derivative alone does not guarantee an extremum.

5 Constrained Optimization (Lagrange Multipliers)

Q5.1. What problem does constrained optimization solve and how is it written?

Ans. Finding the min/max of an objective **subject to a constraint** (harder than the unconstrained case): Minimize/Maximize $J(x)$ subject to $C(x) = 0$.

Q5.2. What is the Lagrangian and the stationarity condition? Define the multiplier.

Ans. Combine objective and constraint: $\pi = J(x) + \lambda C(x)$ and set $\nabla(J(x) + \lambda C(x)) = 0$. Here λ is the **Lagrange multiplier**.

Q5.3. Describe the geometric condition for the constrained optimum (contour picture).

Ans. On a contour plot (a 3-D surface drawn in 2-D), the constrained optimum is where the constraint path is **parallel to a contour** of J , equivalently **perpendicular to the gradient** of J . There the gradient of the objective and the gradient of the constraint are **parallel**: $\nabla J = k \nabla C \Rightarrow \nabla(J + \lambda C) = 0$.

Q5.4. [FLAGGED IN NOTES] Why do we need the bordered Hessian?

Ans. To **determine whether the stationary point is a minimum, maximum, or saddle point**. The gradient/parallel-gradient condition only *locates* a stationary point (it gives the steepest slope for every combination); it does not classify it — the bordered Hessian (second derivatives) does.

Q5.5. What is the structure of the bordered Hessian, and why is its top-left entry zero?

Ans. $H(\pi) = \begin{bmatrix} 0 & \partial C / \partial x \\ (\partial C / \partial x)^\top & \partial^2 \pi / \partial x^2 \end{bmatrix}$: a zero in the corner, the constraint gradient on the border, and the Hessian of π in the block. The (1,1) entry is 0 because $\frac{\partial}{\partial \lambda} \left(\frac{\partial \pi}{\partial \lambda} \right) = \frac{\partial}{\partial \lambda} C(x) = 0$ (C has no λ).

Q5.6. If the constraint is given as $C(x) = g$ (not $= 0$), what do you do first?

Ans. Rewrite it as $C(x) - g = 0$, then form the Lagrangian and proceed.

Q5.7. State the three advantages of the Lagrange-multiplier method.

Ans. (1) Converts a **constrained** problem into an **unconstrained** one; (2) λ carries useful **scale** information; (3) it shows how the **objective changes when the constraint changes** (sensitivity / shadow price).

6 Least-Squares Regression & Correlation

Q6.1. What is the goal of least-squares line fitting and how is error defined?

Ans. Draw the line $y = mx + b$ so all points are **as close as possible** to it (small residuals). Error = real – predicted = $y_i - (mx_i + b)$, and we minimize $SSE = \sum (y_i - mx_i - b)^2$. It is squared so positive/negative errors don't cancel.

Q6.2. [FLAGGED IN NOTES] Derive the least-squares formulas for b and m .

Ans. Set $\partial \text{SSE} / \partial b = 0 \Rightarrow b = \frac{\sum y - m \sum x}{n}$, and $\partial \text{SSE} / \partial m = 0$; substituting b gives $m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2}$. (Sir noted the *proof may be asked*; full algebra is in the Maths & Proofs booklet.)

Q6.3. What does the correlation coefficient r measure, and how do you read its value?

Ans. It describes the **strength and direction of the (linear) relation** between x and y . $r = +1/-1$ = perfect positive/negative line; $r = 0.8, 0.6$ = fairly strong but points scatter off the line; $r = 0$ = no relation. Close to $\pm 1 \Rightarrow$ strong; close to 0 $\Rightarrow$ weak.

Q6.4. What does r^2 mean?

Ans. r^2 is the fraction of variance in the data the model explains: $r^2 = 1$ means the variable/data variance is 100% explained; $r^2 = 0.6$ means 60% is explained.

Q6.5. Does a strong correlation imply causation? Explain.

Ans. No. A **strong correlation does not mean one variable causes the other**. Two things can move together by coincidence, or because a hidden **third factor** influences both (e.g. two things both driven by class being cancelled).

Q6.6. Which running sums must you tabulate to compute m , b , and r ?

Ans. $\sum x$, $\sum y$, $\sum x^2$, $\sum y^2$, $\sum xy$ (build a table).

7 Multiple Linear Regression

Q7.1. Write the multiple-linear-regression model and name every symbol.

Ans. $y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$ — the equation of a hyperplane in n dimensions. w_0 = bias/intercept; $w_1, \dots, w_n$ = weights; $x_1, \dots, x_n$ = features. (1D $\rightarrow$ point, 2D $\rightarrow$ line, 3D $\rightarrow$ plane; without w_0 the fit is stuck through the origin.)

Q7.2. What is the goal, and what is the cost function?

Ans. Goal: find the weights that **minimize the error** between predicted and actual values. Cost = **Mean Squared Error** $J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$, where m = number of samples. Squaring stops $+/ -$ residuals cancelling; the $\frac{1}{2m}$ removes scaling/shifting effects so GD has no unwanted bias.

Q7.3. Write the gradient-descent update for multiple linear regression.

Ans. $w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}$, where α is the learning rate. Initialize $w_0, \dots, w_n$, plug into the update, and iterate until convergence.

Q7.4. List the five assumptions of multiple linear regression.

Ans. (1) **Linearity**; (2) **Independence** (data points independent); (3) **Homoscedasticity** (error variance constant); (4) **Normality** (errors normally distributed); (5) **No multicollinearity** (inputs not linearly dependent on each other).

Q7.5. [FLAGGED IN NOTES] Scenario (10–15 marks): the errors follow a Poisson distribution. Can you apply multiple linear regression? Which assumption is at stake?

Ans. No. If the errors follow a **Poisson distribution** they are not normally distributed, so the **Normality** assumption is violated $\Rightarrow$ multiple linear regression is not appropriate. (Sir noted the exam has scenario-based questions asking whether an assumption is valid.)

Q7.6. What does the homoscedasticity assumption look like when violated, and what does “no multicollinearity” require?

Ans. Homoscedasticity means constant error variance — points don’t fan out and no point sits extremely far; if the spread grows or points are very far, it is violated. **No multicollinearity** requires that the inputs are not linearly related to each other (features not redundant).

8 Lagrange Interpolation

Q8.1. How does interpolation differ from regression?

Ans. Regression finds a **best fit / explanation** and need not pass through every point (it tolerates noise/outliers). **Interpolation passes through all the given points** exactly.

Q8.2. [FLAGGED IN NOTES] Given $(n + 1)$ points, what is the polynomial degree — and why “at most”?

Ans. $(n + 1)$ points $\Rightarrow$ **at most** an n -th order polynomial (degree = #points $- 1$). It is “at most” because if the points happen to be collinear the interpolant reduces to a straight line (lower degree). E.g. 3 points $\rightarrow$ quadratic, 4 points $\rightarrow$ cubic.

Q8.3. Why not just use a very high-order polynomial to fit data?

Ans. A too-high-order polynomial **over-fits**: it tries to fit the noise and chase outliers, giving a wildly oscillating curve. Interpolation is for passing through known points, not for modelling noisy data.

Q8.4. State the Lagrange interpolation formula (compact form).

Ans. $f(x) = \sum_j y_j \prod_{k \neq j} \frac{x - x_k}{x_j - x_k}$. (The index may start from 0 or 1 — just a naming convention.)

Q8.5. [FLAGGED IN NOTES] Outline the proof of the Lagrange formula.

Ans. Assume $f(x) = A_0(x-x_1)\cdots(x-x_n) + A_1(x-x_0)(x-x_2)\cdots(x-x_n) + \dots$. Substituting $x = x_0$ kills every term but the first (each other term has a factor $(x-x_0)$), giving $A_0 = \frac{y_0}{(x_0-x_1)\cdots(x_0-x_n)}$; similarly $x = x_1, \dots, x_n$ give $A_1, \dots, A_n$. Putting the A_k back reconstructs the formula. (Full proof in the Maths & Proofs booklet.)

9 Linear Interpolation

Q9.1. What is linear interpolation and what is its governing equation?

Ans. A special case: fit a straight line $y = mx + b$ through the **two data points that bracket** the target x_m , and read off y_m . Governing equation: $\frac{y_3-y_2}{x_3-x_2} = \frac{y_m-y_2}{x_m-x_2}$ (equal slopes on the same line).

Q9.2. How do you choose which two points to use for a query x ?

Ans. Always use the two tabulated points that **surround (bracket)** the query. For x_m between x_2 and x_3 , use $(x_2, y_2), (x_3, y_3)$; for an x_n between x_1 and x_2 , use $(x_1, y_1), (x_2, y_2)$, giving $\frac{y_2-y_1}{x_2-x_1} = \frac{y_n-y_1}{x_n-x_1}$.

10 Newton's Divided-Difference Interpolation (v2 notes)

Q10.1. What does Newton's polynomial compute, and what form does it take?

Ans. It computes a polynomial that **interpolates a set of points** $\{(x_i, y_i)\}$ (same job as Lagrange), of the form $P_n(x) = a_0 + a_1(x-x_0) + a_2(x-x_0)(x-x_1) + \cdots + a_n \prod_{i=0}^{n-1} (x-x_i)$. It grows term-by-term: 1 point $\rightarrow$ constant, 2 points $\rightarrow$ line (P_1), 3 points $\rightarrow$ quadratic (P_2), and so on.

Q10.2. What are the coefficients a_0, a_1, a_2 , and what is a “divided difference”?

Ans. Substituting the nodes one at a time gives $a_0 = y_0$, $a_1 = \frac{y_1-y_0}{x_1-x_0} = f[x_1, x_0]$ (1st-order divided difference), and $a_2 = f[x_2, x_1, x_0]$ (2nd-order). A **divided difference** is this recursive slope-of-slopes quantity that forms each coefficient.

Q10.3. [FLAGGED IN NOTES] State the recurrence relation and note where it is used.

Ans. $f[x_0, x_1, \dots, x_{n+1}] = \frac{f[x_1, \dots, x_{n+1}] - f[x_0, \dots, x_n]}{x_{n+1} - x_0}$. Sir noted **Newton uses this recurrence to build a table**, and *you will have to build that table in the exam*. Memory trick: “(drop-first group – drop-last group) over (last x – first x)”.

Q10.4. What is the key advantage of Newton's form over Lagrange's (the "recalculation-free property")?

Ans. When a **new data point is added**, the existing coefficients $a_0, a_1, \dots$ stay the **same**; you only compute one new term and append it. Lagrange interpolation would force you to recompute every basis polynomial from scratch. In the table, you just add one new bottom row — old numbers are untouched.

Q10.5. Outline the proof that the divided-difference recurrence is valid.

Ans. Let P_n interpolate $x_0 \rightarrow x_n$ and L_n interpolate $x_1 \rightarrow x_{n+1}$ (both n -th order). Form $P_{n+1}(x) = \frac{(x-x_0)L_n(x) + (x_{n+1}-x)P_n(x)}{x_{n+1}-x_0}$. At $x = x_0$ it gives y_0 ; at $x = x_{n+1}$ it gives y_{n+1} ; at any interior x_k both agree so it gives y_k — hence it interpolates all points. Equating the coefficient of the highest power x^{n+1} on both sides yields the recurrence.

Q10.6. What happens when the nodes are equally spaced (equi-difference)?

Ans. If the x 's have a constant step h ($x_2 - x_1 = h$, $x_3 - x_1 = 2h$, $\dots$ — e.g. census years 2000, 2001, $\dots$), the divided differences simplify and you can use **forward/backward difference** formulas instead.

11 Spline Interpolation (v2 notes)

Q11.1. What is spline interpolation, and how does it differ from Newton/Lagrange?

Ans. A spline is **local interpolation**: it uses a **subset of the data** and joins several short, low-order pieces (one small curve per interval). Newton and Lagrange are **global**: all points feed one big polynomial, where $(n+1)$ points force an n -th order polynomial.

Q11.2. [FLAGGED IN NOTES] What is the problem with global interpolation (Runge's Phenomenon)?

Ans. As data points increase, the single polynomial's **order rises**, and a high-order polynomial **oscillates badly — worst at the edges** of equally-spaced points; the error at the edges grows. This blow-up is **Runge's Phenomenon** (classic example: interpolating $\frac{1}{1+x^2}$, where the fit wiggles and can even go negative although the function never does). **Solution:** use a subset of points $\rightarrow$ local interpolation (splines).

Q11.3. Describe the linear spline and its weakness.

Ans. Connect consecutive data points with straight lines, e.g. $P_1(x) = y_1 + \frac{y_2 - y_1}{x_2 - x_1}(x - x_1)$, and use the piece whose interval contains the query. **Weakness: no curvature** — the joints are sharp corners, so it is not smooth. This motivates quadratic/cubic splines.

Q11.4. For a quadratic spline, how many unknowns are there and how are the equations obtained?

Ans. Each piece $P_i = a_i x^2 + b_i x + c_i$ has 3 coefficients, so with n splines there are **$3n$ unknowns**. Equations: (1) pass through known points $\Rightarrow 2n$; (2) first-derivative match at interior points (smoothness) $\Rightarrow (n-1)$; (3) one assumption, $f'' = 0$ at one end $\Rightarrow 1$. Total = $2n + (n-1) + 1 = 3n$.

Q11.5. What does “smoothness” mean at a spline joint?

Ans. At a joining point both curves have the **same tangent** (matching first derivative), so the transition looks smooth; if the tangents differ there is a **kink** (not smooth).

Q11.6. For a cubic spline: unknowns, what it guarantees, benefits, and the equations?

Ans. Each piece $P_i = a_i x^3 + b_i x^2 + c_i x + d_i$ has 4 coefficients $\Rightarrow$ **$4n$ unknowns**. It **ensures continuity and smoothness**. Benefits: (1) good function fit, (2) reduces oscillation, (3) smoothness and continuity. Equations: (1) through points $\Rightarrow 2n$; (2) 1st-derivative match at interiors $\Rightarrow (n-1)$; (3) 2nd-derivative match at interiors (maintains continuity) $\Rightarrow (n-1)$; (4) boundary assumptions $\Rightarrow 2$. Total = $4n$.

Q11.7. Contrast the two cubic-spline boundary assumptions: natural vs clamped.

Ans. Natural cubic: the curvature is zero at both ends, $P_1''(x_{\text{first}}) = 0$ and $P_n''(x_{\text{last}}) = 0$. **Clamped cubic:** you are given **extra data** — the slope $\frac{dy}{dx}$ at the two ends. Either way, every spline’s degrees of freedom must be pinned down so the $4n$ unknowns are solvable.

Q11.8. What is the trade-off in choosing a spline order?

Ans. Higher order $\Rightarrow$ more smoothness, but more computation cost and more error. Linear is cheapest but has corners; cubic is smooth and continuous but costlier. It is a smoothness-vs-cost trade-off.

Q11.9. [FLAGGED IN NOTES] What does the CT syllabus cover?

Ans. Sir wrote the CT covers **Theory, Concept, and Math** — so be ready to state definitions (theory), explain the reasoning/why (concept), and carry out the computations and derivations (math).

Covered end-to-end from sir’s notes (both v1 and v2) — if you can answer all of these, you’ve covered the theory.